API Testing with Playwright

LESSON SPECIFICATIONS & ROADMAP

• Track: Playwright Automation with JavaScript + AI • Level: Advanced
• Estimated Effort: 2.5–3 Hours • Prerequisites: Days 1–12 concepts (HTTP basics, JavaScript async/await, test runner, asser
• Runtime: Pure Modern JavaScript (.js) • Runner: @playwright/test

Combine backend REST API validation with browser UI workflows using Playwright's built-in request fixture. Seed test data in milliseconds, verify database mutations, and perform fast headless API regression.

Day 13 Learning Objectives

'Master HTTP methods (GET, POST, PUT, PATCH, DELETE) and status code categories (2xx, 4xx, 5xx).
'Use Playwright's built-in request fixture and APIRequestContext for direct HTTP communication.
'Validate API response headers, status codes, JSON response bodies, and schema structures.
'Author negative API tests verifying 400 Bad Request, 401 Unauthorized, and 404 Not Found responses.
'Implement hybrid UI + API workflows: seed test data via API in 50ms, verify via UI, and clean up via API.
'Verify backend database mutations via API after completing user actions in the browser UI.
'Avoid destructive API requests against production and maintain isolated test environments.

Modern web applications communicate with backend services over the **HTTP (Hypertext Transfer Protocol)** protocol. Testing the API layer directly provides faster execution, earlier defect discovery, and simplified test data preparation.

Core HTTP Methods
- **GET: Retrieve resources (idempotent, safe, no request body).**
- **POST: Create new resources (returns 201 Created with generated ID).**
- **PUT: Replace an entire existing resource.**
- **PATCH: Partially update specific fields on an existing resource.**
- **DELETE: Remove a resource (returns 200 OK or 204 No Content).**

HTTP Status Code Categories
- **2xx Success: 200 OK (read/update), 201 Created (insert), 204 No Content (deletion).**
- **4xx Client Errors: 400 Bad Request (schema invalid), 401 Unauthorized (missing auth), 403 Forbidden (insufficient permissions), 404 Not Found (resource absent).**
- **5xx Server Errors: 500 Internal Server Error (unhandled backend exception), 502 Bad Gateway, 503 Service Unavailable.**

Request Headers & Payload
Every API request can specify headers such as:
- `Content-Type: application/json`
- `Authorization: Bearer <token>`

Standard REST Request Structure (A POST request creating a task resource.)

```
POST /api/tasks HTTP/1.1
Host: api.rootaccess.org
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{
  "title": "Complete Playwright Day 13",
  "priority": "High"
}
```

Unlike Selenium, which requires external HTTP libraries like Axios or RestAssured, Playwright includes a built-in, native HTTP client: **Playwright APIRequestContext**!

The `request` Fixture
In Playwright Test, you can destructure `request` directly into your test arguments:
```javascript
const { test, expect } = require('@playwright/test');
```

```
test('fetch user profile via API', async ({ request }) => {
  const response = await request.get('/api/users/me', {
    headers: {
      Authorization: `Bearer ${process.env.API_TOKEN}`,
    },
  });

  // Verify HTTP status code
  expect(response.status()).toBe(200);
  expect(response.ok()).toBeTruthy();

  // Parse JSON response body
  const body = await response.json();
  expect(body.email).toBe('student.automation@rootaccess.org');
  expect(body.role).toBe('student');
});
```

Executing POST, PUT, and DELETE Requests

```
test('CRUD lifecycle on task API', async ({ request }) => {
  // CREATE (POST)
  const createRes = await request.post('/api/tasks', {
    data: {
      title: 'Audit Cypress vs Playwright',
      status: 'pending',
    },
  });
  expect(createRes.status()).toBe(201);
  const created = await createRes.json();
  const taskId = created.id;

  // UPDATE (PATCH)
  const updateRes = await request.patch(`/api/tasks/${taskId}`, {
    data: { status: 'completed' },
  });
  expect(updateRes.status()).toBe(200);

  // DELETE
  const deleteRes = await request.delete(`/api/tasks/${taskId}`);
  expect(deleteRes.status()).toBe(204);
});
```

Negative API Testing (Testing that invalid requests are rejected with 400 Bad Request.)

```
test('rejects task creation when title is missing', async ({ request }) => {
  const response = await request.post('/api/tasks', {
    data: { priority: 'Low' }, // missing required title
  });

  expect(response.status()).toBe(400);
  const body = await response.json();
  expect(body.error).toContain('Title is required');
});
```

A comprehensive API test verifies more than just `status === 200`. It asserts that the contract between frontend and backend remains intact.

4 Levels of API Assertions

1. **Status Code: Verify exact response codes (`200`, `201`, `400`, `401`, `404`).**
2. **Response Headers: Verify `content-type` includes `application/json`.**
3. **Payload Structure & Types: Verify required properties exist and possess correct JavaScript data types (`typeof body.id === 'string'`).**
4. **Business Logic & State Mutation: Verify database values reflect the requested changes.**

```javascript
test('validate customer profile schema and data integrity', async ({ request }) => {
  const res = await request.get('/api/customers/101');
  expect(res.status()).toBe(200);
```

```
  // Validate headers
  expect(res.headers()['content-type']).toContain('application/json');

  const customer = await res.json();

  // Validate field presence and types
  expect(customer).toHaveProperty('id');
  expect(customer).toHaveProperty('email');
  expect(customer).toHaveProperty('tier');
  expect(typeof customer.isActive).toBe('boolean');
  expect(Array.isArray(customer.enrolledCourses)).toBe(true);
});
```

Verifying Array Responses (Asserting that a list endpoint returns an array with expected length and items.)

```
test('list courses returns active catalog array', async ({ request }) => {
  const res = await request.get('/api/courses');
  const courses = await res.json();

  expect(Array.isArray(courses)).toBe(true);
  expect(courses.length).toBeGreaterThanOrEqual(1);
  expect(courses[0]).toHaveProperty('slug');
});
```

The real superpowers of Playwright emerge when you **combine UI interactions with API requests** in the same test!

1. Seeding Test Data via API (Blazing Fast Setup)
Imagine a UI test that tests "Archiving an Order".
- **Slow UI-Only Approach: Log in (5s), search catalog (3s), add to cart (2s), enter shipping address (4s), enter credit card (3s), submit order (4s), then click Archive (2s). Total: **23 seconds!****
- **Hybrid API Approach:**
1. Call `await request.post('/api/orders', { data: ... })` -> **50 milliseconds!**
2. In UI: `await page.goto('/orders/' + orderId)` and click Archive -> **2 seconds!**
Result: 10x faster execution and 90% fewer UI points of failure!**

2. Verifying Backend State After a UI Action
When a user clicks "Enroll in Course" in the UI:
1. Perform the click in the browser: `await page.getByRole('button', { name: 'Enroll' }).click();`
2. Assert UI feedback: `await expect(page.getByText('Enrolled')).toBeVisible();`
3. Verify backend database record via API:
```javascript
const res = await request.get('/api/users/me/enrollments');
const body = await res.json();
expect(body.enrolledCourses).toContain('playwright-automation');
```

3. Fast Teardown via API
Delete created records via `await request.delete('/api/items/' + id)` in `afterEach` to guarantee database cleanup without navigating through complex deletion UI modals.

Hybrid Seed -> UI Action -> API Verification (A complete hybrid Playwright test.)

```
test('create task via API and verify in UI', async ({ page, request }) => {
  // 1. Seed via API (50ms)
  const res = await request.post('/api/tasks', { data: { title: 'Seed Task' } });
  const task = await res.json();

  // 2. Interact via UI
  await page.goto('/tasks');
  await expect(page.getByText('Seed Task')).toBeVisible();

  // 3. Delete via API in teardown
  await request.delete('/api/tasks/' + task.id);
});
```

requirements.

Assignment Starter Code (Day 13)

```
// tests/task_management_api.spec.js
const { test, expect } = require('@playwright/test');

test.describe('Task Management REST API & Hybrid UI Suite', () => {

  let createdTaskId = null;
  const testTaskTitle = `Automation Task ${Date.now()}`;

  // 1. CREATE (POST) - Create a new task via REST API
  test('TC-API-01: POST /api/tasks creates task with 201 Created and valid schema', async ({ request }) => {
    const response = await request.post('/api/tasks', {
      data: {
        title: testTaskTitle,
        priority: 'High',
        completed: false,
      },
    });

    // Assert status code
    expect(response.status()).toBe(201);
    expect(response.ok()).toBeTruthy();
```

DAY 13 INTERVIEW PREPARATION QUESTION BANK

Q1. How do you perform API testing in Playwright, and what is the role of the request fixture?

[Difficulty: Beginner] Evaluation Focus: API testing capabilities and architecture in Playwright.

Model Answer: Playwright provides a built-in `request` fixture (an instance of `APIRequestContext`) that allows tests to send HTTP requests directly from Node.js without launching a browser. It supports all standard verbs—`request.get()`, `post()`, `put()`, `patch()`, and `delete()`. Tests can inspect `response.status()`, `response.headers()`, `response.json()`, and `response.text()`. Because no browser engine is launched for pure API tests, they execute in single-digit milliseconds, making them ideal for smoke tests and contract validation.

Follow-Up Question: "How do you share cookies or tokens between the browser context and the request fixture?"

Q2. What is a "hybrid" test in Playwright, and what benefits does it offer over pure UI-only tests?

[Difficulty: Intermediate] Evaluation Focus: Hybrid testing strategy, test data seeding, and performance optimization.

Model Answer: A hybrid test combines browser UI actions with direct backend API calls. Typically, we use the `request` fixture to rapidly seed prerequisite data (such as creating a user account, generating an order, or populating a cart) in 50 milliseconds via API. The test then navigates directly to the target page via `page.goto()` to test the specific UI feature under test. After the UI action completes, we can verify database persistence via an API GET call and clean up via an API DELETE call. This reduces test execution duration from 30 seconds to 2 seconds and eliminates UI flakiness in setup steps.

Follow-Up Question: "How does API teardown improve test independence in continuous execution?"

Q3. How do you test negative API scenarios (e.g., 400 Bad Request or 401 Unauthorized) in Playwright?

[Difficulty: Intermediate] Evaluation Focus: Negative testing, validation schemas, and error handling.

Model Answer: In Playwright, we intentionally construct malformed or unauthorized HTTP requests using the `request` fixture. For example, to test 400 Bad Request, we omit required payload fields or pass invalid data types. We assert `expect(response.status()).toBe(400)` and inspect the parsed JSON body with `expect(body.error).toContain("Required field missing")`. To test 401 Unauthorized, we send requests without the `Authorization` header and assert `expect(response.status()).toBe(401)`. Playwright does not throw on 4xx/5xx responses, allowing you to assert error status codes cleanly.

Follow-Up Question: "Why does Playwright not throw exceptions on 4xx and 5xx HTTP responses by default?"

Q4. How do you validate an API response schema to protect against backend breaking changes?

[Difficulty: Intermediate] Evaluation Focus: Contract verification, schema validation, and data typing.

Model Answer: We validate the structure and data types of the returned JSON body. We assert that required keys are present using `expect(body).toHaveProperty("key")`, verify data types using `typeof body.id === "string"`, and confirm array structures with `Array.isArray(body.items)`. For strict enterprise contract testing, we can integrate lightweight JSON schema validators like Ajv or Zod to validate the response against an OpenAPI/Swagger contract.

Follow-Up Question: "What is the risk of testing only exact values rather than structural schemas?"

Q5. How do you configure base URLs and authentication tokens globally for API tests in Playwright?

[Difficulty: Scenario-Based] Evaluation Focus: playwright.config.js configuration and environment variables.

Model Answer: We configure `baseURL` inside `use` in `playwright.config.js`, pointing to our API server (e.g. `baseURL: process.env.API_BASE_URL || "https://api.rootaccess.org"`). Relative paths like `request.get("/api/tasks")` automatically inherit this base URL. For authentication headers, we can configure `extraHTTPHeaders` in `playwright.config.js` to automatically attach `Authorization: Bearer ${process.env.API_TOKEN}` to all outgoing requests, keeping individual test files clean and focused.

Follow-Up Question: "How do you handle tests that require switching between Admin and Student API tokens?"

1. **Which built-in Playwright fixture enables direct HTTP API testing without launching a browser?**
Key Concept: The request fixture (an instance of APIRequestContext) provides native HTTP request capabilities directly in Playwright Test.

2. **Which method reads the parsed JSON body of an API response in Playwright?**
Key Concept: await response.json() asynchronously parses the response stream and returns the parsed JavaScript object.

3. **What does response.ok() return in Playwright?**
Key Concept: response.ok() returns a boolean indicating whether the HTTP status code is considered successful (status in the range 200–299).

4. **What is the primary benefit of "hybrid" UI + API automation in Playwright?**
Key Concept: Hybrid testing leverages the speed and reliability of APIs for prerequisite setup and teardown while reserving browser automation for true user-facing interactions.

5. **When an endpoint returns a 404 Not Found, what does Playwright's request method do?**
Key Concept: Playwright does not throw errors on 4xx/5xx responses; it returns the APIResponse object so you can cleanly assert error status codes and error payloads.

6. **Which HTTP method should be used to update only specific fields of an existing task record?**
Key Concept: HTTP PATCH is used for partial modifications of an existing resource, whereas PUT typically replaces the entire resource.

7. **How do you pass a JSON payload when calling request.post() in Playwright?**
Key Concept: In Playwright, the data property in the options object automatically serializes JavaScript objects to JSON and sets the Content-Type header to application/json.

8. **Why is it important to perform API teardown (e.g. DELETE /api/items/123) in an afterEach hook?**
Key Concept: Teardown hooks ensure that test environments remain clean, isolated, and free of orphaned test data that could cause collisions in future test runs.

9. **How can you verify that a UI action (e.g. clicking "Delete Task") actually deleted the record on the server?**
Key Concept: Querying the backend API directly after a UI action provides ironclad proof that the server-side database mutation succeeded, not just that the UI element was hidden.

10. **Where can you define global extraHTTPHeaders (such as Authorization tokens) for all API requests in Playwright?**
Key Concept: Setting extraHTTPHeaders in playwright.config.js ensures all outgoing request fixture calls automatically include default headers like authorization tokens or content types.